TOPIC 2

Experiment 1

Aim:
As specified in the uploaded experiment sheet.

Algorithm:
1. Read input.
2. Apply the required algorithm.
3. Display the result.

Pseudocode:
START
Read input
Process using algorithm
Print output
STOP

Python Program:

```
def selection_sort(arr):

  n = len(arr)


  for i in range(n):

    min_index = i


    for j in range(i + 1, n):

      if arr[j] < arr[min_index]:

        min_index = j


    arr[i], arr[min_index] = arr[min_index], arr[i]


  return arr
```

```
# Test Cases
test_cases = [
    [],
    [1],
    [7, 7, 7, 7],
    [-5, -1, -3, -2, -4]
]

for arr in test_cases:
    print("Input :", arr)
    print("Output:", selection_sort(arr.copy()))
    print()
```

Sample Input:
Test Case 1:

[]

Test Case 2:

[1]

Test Case 3:

[7, 7, 7, 7]

Test Case 4:

[-5, -1, -3, -2, -4]

Sample Output:
Test Case 1:

[]

Test Case 2:

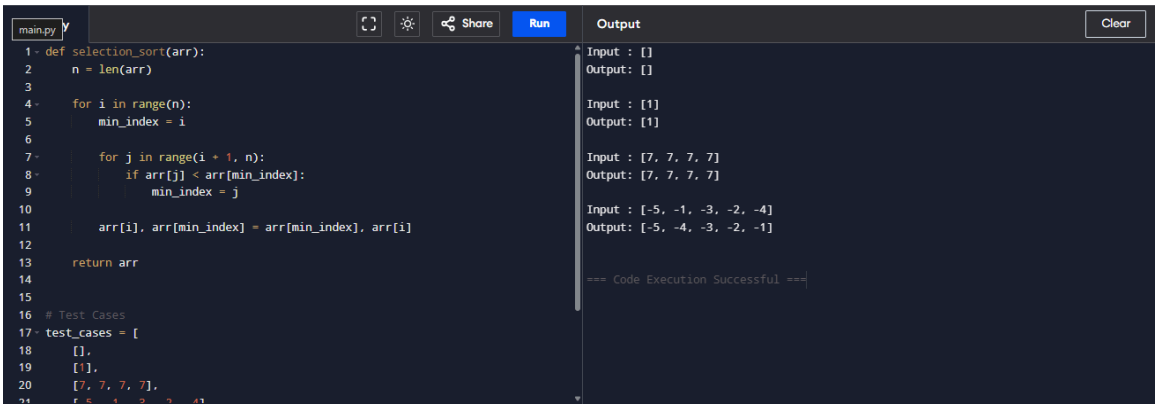
[1]

Test Case 3:

[7, 7, 7, 7]

Test Case 4:

[-5, -4, -3, -2, -1]



Result:
The program was executed successfully and the expected output was obtained.

Experiment 2

Aim:
As specified in the uploaded experiment sheet.

Algorithm:
1. Read input.
2. Apply the required algorithm.
3. Display the result.

Pseudocode:
START
Read input
Process using algorithm
Print output
STOP

Python Program:

```
def selection_sort(arr):
    n = len(arr)

    for i in range(n):
        min_index = i

        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j

        arr[i], arr[min_index] = arr[min_index], arr[i]

    return arr



# Test Cases
arrays = [
    [5, 2, 9, 1, 5, 6],
    [10, 8, 6, 4, 2],
```

```
    [1, 2, 3, 4, 5]
]

for arr in arrays:
    print("Input :", arr)
    print("Sorted:", selection_sort(arr.copy()))
    print()
```

Sample Input:
Test Case 1:

[5, 2, 9, 1, 5, 6]

Test Case 2:

[10, 8, 6, 4, 2]

Test Case 3:

[1, 2, 3, 4, 5]

Sample Output:
Test Case 1:

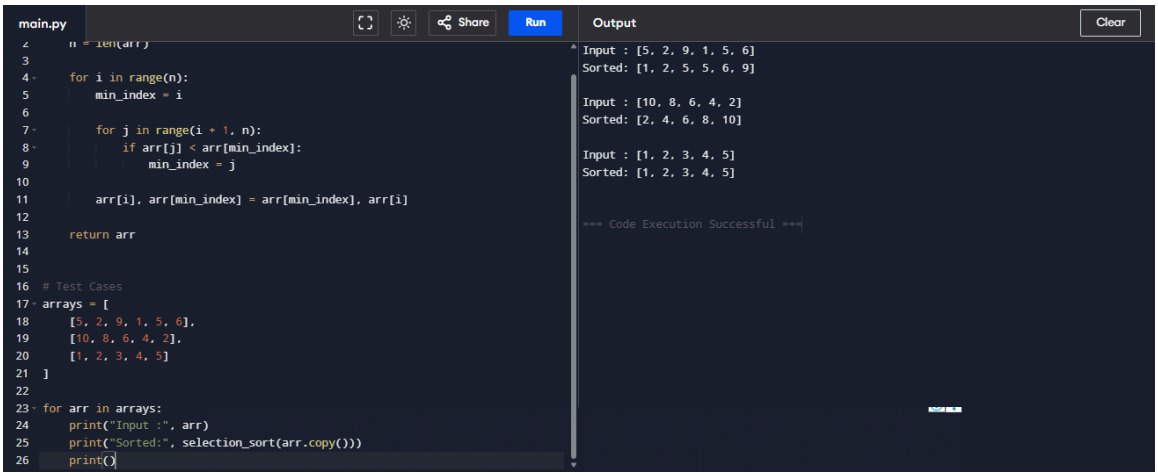
[1, 2, 5, 5, 6, 9]

Test Case 2:

[2, 4, 6, 8, 10]

Test Case 3:

[1, 2, 3, 4, 5]



Result:
The program was executed successfully and the expected output was obtained.

Experiment 3

Aim:
As specified in the uploaded experiment sheet.

Algorithm:
1. Read input.
2. Apply the required algorithm.
3. Display the result.

Pseudocode:
START
Read input
Process using algorithm
Print output
STOP

Python Program:

```
def bubble_sort(arr):
    n = len(arr)

    for i in range(n):
        swapped = False

        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True

        if not swapped:
            break

    return arr



# Test Cases
tests = [
    [64, 25, 12, 22, 11],
    [29, 10, 14, 37, 13],
```

```
    [3, 5, 2, 1, 4],
    [1, 2, 3, 4, 5],
    [5, 4, 3, 2, 1]
]

for arr in tests:
    print("Input :", arr)
    print("Sorted:", bubble_sort(arr.copy()))
    print()
```

Sample Input:
Test Case 1:

[64, 25, 12, 22, 11]

Test Case 2:

[29, 10, 14, 37, 13]

Test Case 3:

[3, 5, 2, 1, 4]

Test Case 4:

[1, 2, 3, 4, 5]

Test Case 5:

[5, 4, 3, 2, 1]

Sample Output:
Test Case 1:

[11, 12, 22, 25, 64]

Test Case 2:

[10, 13, 14, 29, 37]

Test Case 3:

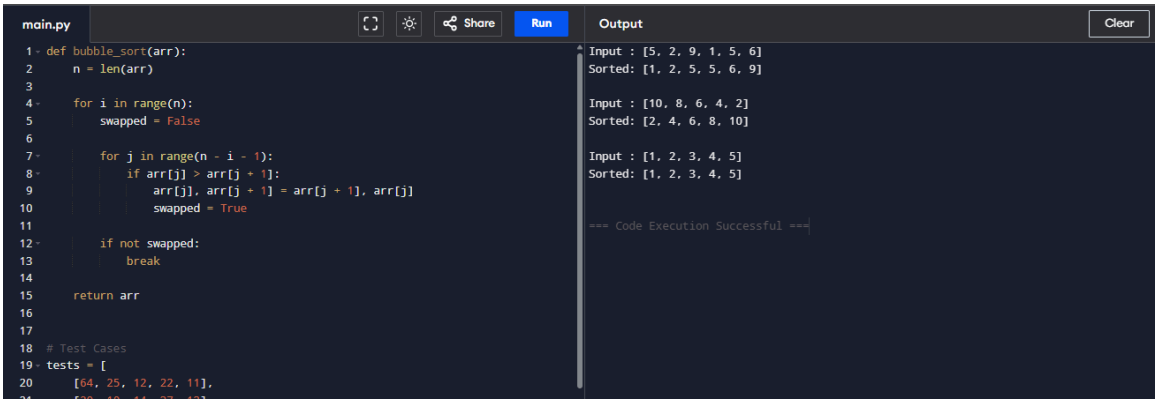
[1, 2, 3, 4, 5]

Test Case 4:

[1, 2, 3, 4, 5]

Test Case 5:

[1, 2, 3, 4, 5]



Result:
The program was executed successfully and the expected output was obtained.

Experiment 4

Aim:
As specified in the uploaded experiment sheet.

Algorithm:
1. Read input.

2. Apply the required algorithm.
3. Display the result.

Pseudocode:
START
Read input
Process using algorithm
Print output
STOP

Python Program:

```
def insertion_sort(arr):


    for i in range(1, len(arr)):

        key = arr[i]

        j = i - 1


        while j >= 0 and arr[j] > key:

            arr[j + 1] = arr[j]

            j -= 1


        arr[j + 1] = key


    return arr




# Test Cases

arrays = [

    [3, 1, 4, 1, 5, 9, 2, 6, 5, 3],

    [5, 5, 5, 5, 5],

    [2, 3, 1, 3, 2, 1, 1, 3]
```

```
]

for arr in arrays:
    print("Input :", arr)
    print("Sorted:", insertion_sort(arr.copy()))
    print()
```

Sample Input:
Test Case 1:

[64, 25, 12, 22, 11]

Test Case 2:

[29, 10, 14, 37, 13]

Test Case 3:

[3, 5, 2, 1, 4]

Test Case 4:

[1, 2, 3, 4, 5]

Test Case 5:

[5, 4, 3, 2, 1]

Sample Output:
Test Case 1:

[11, 12, 22, 25, 64]

Test Case 2:

[10, 13, 14, 29, 37]

Test Case 3:

[1, 2, 3, 4, 5]

Test Case 4:

[1, 2, 3, 4, 5]

Test Case 5:

[1, 2, 3, 4, 5]

main.py

```python
def insertion_sort(arr):

    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1

        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1

        arr[j + 1] = key

    return arr


# Test Cases
arrays = [
    [3, 1, 4, 1, 5, 9, 2, 6, 5, 3],
    [5, 5, 5, 5, 5],
    [2, 3, 1, 3, 2, 1, 1, 3]
]
```

Output

```
Input : [3, 1, 4, 1, 5, 9, 2, 6, 5, 3]
Sorted: [1, 1, 2, 3, 3, 4, 5, 5, 6, 9]

Input : [5, 5, 5, 5, 5]
Sorted: [5, 5, 5, 5, 5]

Input : [2, 3, 1, 3, 2, 1, 1, 3]
Sorted: [1, 1, 1, 2, 2, 3, 3, 3]


=== Code Execution Successful ===
```

Result:
The program was executed successfully and the expected output was obtained.

Experiment 5

Aim:
As specified in the uploaded experiment sheet.

Algorithm:
1. Read input.
2. Apply the required algorithm.
3. Display the result.

Pseudocode:
START
Read input
Process using algorithm
Print output
STOP

Python Program:

```
def findKthPositive(arr, k):
    current = 1
    index = 0
    while k > 0:
        if index < len(arr) and arr[index] == current:
            index += 1
        else:
            k -= 1
            if k == 0:
                return current
        current += 1
# Test Cases
print("Example 1")
arr = [2, 3, 4, 7, 11]
k = 5
print("Output:", findKthPositive(arr, k))
print()
print("Example 2")
arr = [1, 2, 3, 4]
k = 2
print("Output:", findKthPositive(arr, k))
```

Sample Input:
Test Case 1:

Array = [2, 3, 4, 7, 11]

K = 5

Test Case 2:

Array = [1, 2, 3, 4]

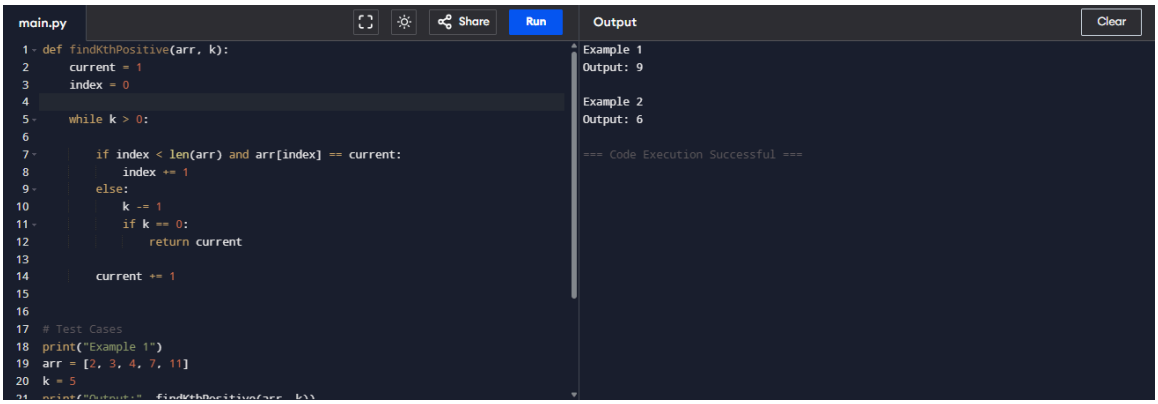
K = 2

Sample Output:
Test Case 1:

9

Test Case 2:

6



Result:
The program was executed successfully and the expected output was obtained.

Experiment 6

Aim:
As specified in the uploaded experiment sheet.

Algorithm:
1. Read input.
2. Apply the required algorithm.
3. Display the result.

Pseudocode:
START
Read input
Process using algorithm
Print output
STOP

Python Program:

```
def brute_force(text, pattern):

    n = len(text)

    m = len(pattern)



    for i in range(n - m + 1):

        j = 0

        while j < m and text[i + j] == pattern[j]:

            j += 1



        if j == m:

            return i



    return -1




text = input("Enter Text: ")

pattern = input("Enter Pattern: ")



index = brute_force(text, pattern)



if index != -1:
```

print("Pattern found at index", index)

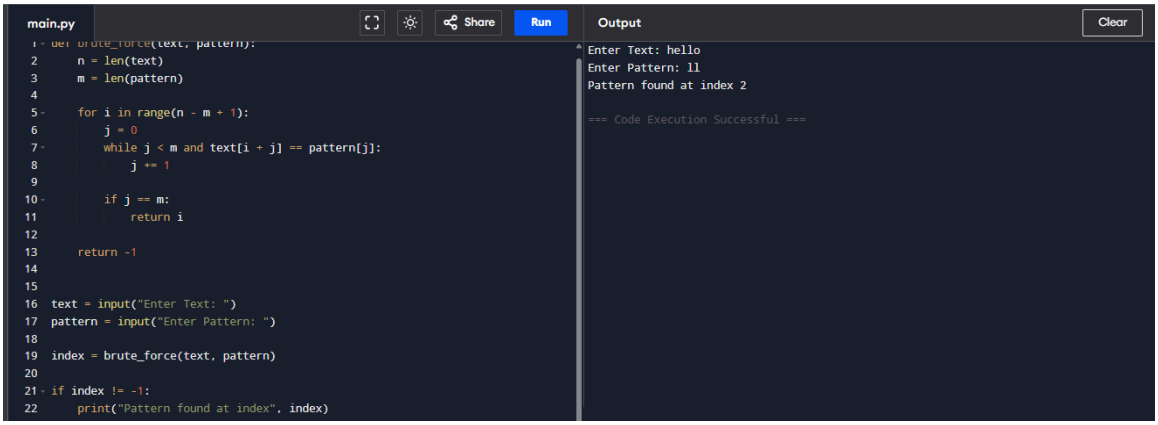
else:

print("Pattern not found")

Sample Input:
Enter Text: AABAACAADAABAABA

Enter Pattern: AABA

Sample Output:
Pattern found at index 0



Result:
The program was executed successfully and the expected output was obtained.

Experiment 7

Aim:
As specified in the uploaded experiment sheet.

Algorithm:
1. Read input.
2. Apply the required algorithm.
3. Display the result.

Pseudocode:
START
Read input
Process using algorithm
Print output
STOP

Python Program:

```
import math

n = int(input("Enter number of points: "))

points = []

for i in range(n):

   x, y = map(int, input().split())

   points.append((x, y))

min_dist = float('inf')

pair = ()

for i in range(n):

   for j in range(i + 1, n):

      d = math.sqrt((points[i][0] - points[j][0]) ** 2 +

               (points[i][1] - points[j][1]) ** 2)

      if d < min_dist:

         min_dist = d

         pair = (points[i], points[j])

print("Closest Pair:", pair)

print("Minimum Distance:", round(min_dist, 2))
```

Sample Input:
Enter number of points:

4

2 3

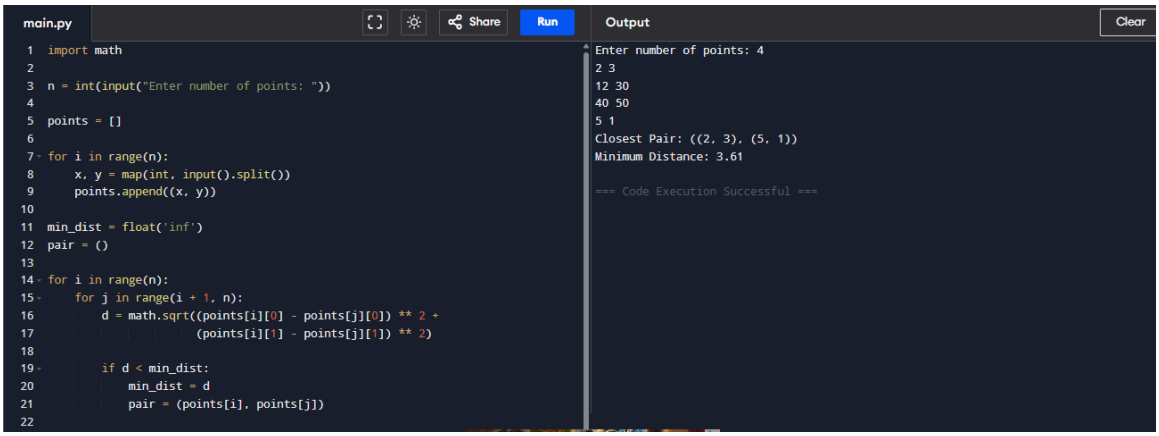
12 30

40 50

5 1

Sample Output:
Closest Pair: ((2, 3), (5, 1))

Minimum Distance: 3.61



Result:
The program was executed successfully and the expected output was obtained.

Experiment 8

Aim:
As specified in the uploaded experiment sheet.

Algorithm:
1. Read input.
2. Apply the required algorithm.
3. Display the result.

Pseudocode:
START

Read input
Process using algorithm
Print output
STOP

Python Program:

```
from itertools import permutations


graph = [
   [0, 10, 15, 20],
   [10, 0, 35, 25],
   [15, 35, 0, 30],
   [20, 25, 30, 0]
]


n = len(graph)
vertices = list(range(1, n))
min_cost = float('inf')
for path in permutations(vertices):
   cost = 0
   k = 0


   for j in path:
      cost += graph[k][j]
      k = j
   cost += graph[k][0]
   min_cost = min(min_cost, cost)
print("Minimum Cost:", min_cost)
```

Sample Input:
Distance Matrix

0 10 15 20

10 0 35 25

15 35 0 30

20 25 30 0

Sample Output:
Minimum Cost: 80

main.py

```python
from itertools import permutations

graph = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]

n = len(graph)

vertices = list(range(1, n))

min_cost = float('inf')

for path in permutations(vertices):
    cost = 0
    k = 0

    for j in path:
        cost += graph[k][j]
        k = j

    cost += graph[k][0]
```

Share Run Output Clear

```
Minimum Cost: 80

=== Code Execution Successful ===
```

Result:
The program was executed successfully and the expected output was obtained.

Experiment 9

Aim:
As specified in the uploaded experiment sheet.

Algorithm:
1. Read input.
2. Apply the required algorithm.
3. Display the result.

Pseudocode:
START
Read input
Process using algorithm

Print output
STOP

Python Program:

```
def orientation(p, q, r):
    return (q[1]-p[1])*(r[0]-q[0]) - (q[0]-p[0])*(r[1]-q[1])


def convex_hull(points):
    if len(points) < 3:
        return []

    left = 0
    for i in range(1, len(points)):
        if points[i][0] < points[left][0]:
            left = i

    hull = []

    p = left

    while True:
        hull.append(points[p])

        q = (p + 1) % len(points)

        for i in range(len(points)):
            if orientation(points[p], points[i], points[q]) < 0:
                q = i
```

```
        p = q

        if p == left:
            break

    return hull

points = [(0,3),(2,2),(1,1),(2,1),(3,0),(0,0),(3,3)]

print("Convex Hull:")
print(convex_hull(points))
```

Sample Input:
Points:

(0,3)

(2,2)

(1,1)

(2,1)

(3,0)

(0,0)

(3,3)

Sample Output:
Convex Hull:

[(0, 3), (0, 0), (3, 0), (3, 3)]

Result:
The program was executed successfully and the expected output was obtained.

Experiment 10

Aim:
As specified in the uploaded experiment sheet.

Algorithm:
1. Read input.
2. Apply the required algorithm.
3. Display the result.

Pseudocode:
START
Read input
Process using algorithm
Print output
STOP

Python Program:

```
from itertools import permutations


cost = [
   [9,2,7],
   [6,4,3],
   [5,8,1]
]


n = len(cost)


workers = range(n)


min_cost = float('inf')
best = None
for perm in permutations(workers):
   total = 0
   for job in workers:
      total += cost[perm[job]][job]
   if total < min_cost:
      min_cost = total
      best = perm
print("Minimum Cost:", min_cost)
print("Assignment:", best)
```

Sample Input:
Cost Matrix

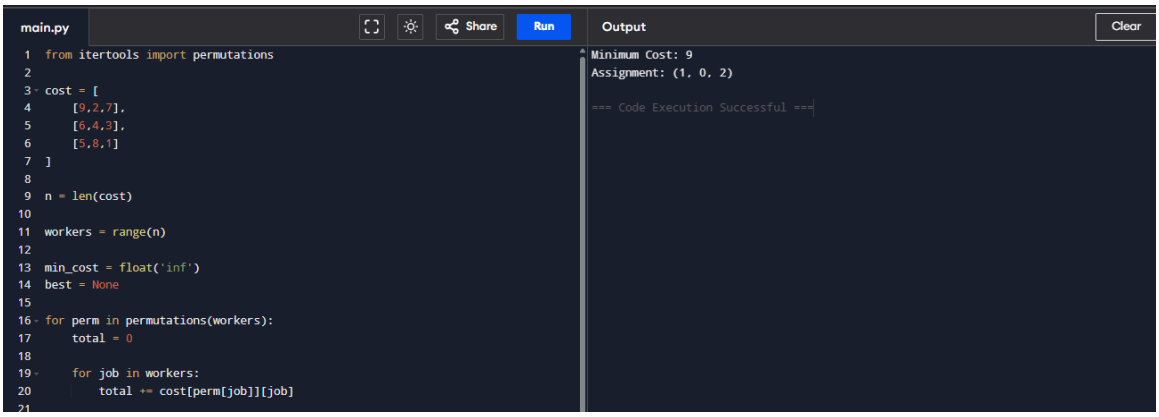
9 2 7

6 4 3

5 8 1

Sample Output:
Minimum Cost: 9

Assignment: (1, 0, 2)



Result:
The program was executed successfully and the expected output was obtained.

Experiment 11

Aim:
As specified in the uploaded experiment sheet.

Algorithm:
1. Read input.
2. Apply the required algorithm.
3. Display the result.

Pseudocode:
START
Read input
Process using algorithm
Print output
STOP

Python Program:

```
def knapsack(weights, values, capacity):
    n = len(values)

    dp = [[0 for _ in range(capacity + 1)] for _ in range(n + 1)]

    for i in range(1, n + 1):
        for w in range(1, capacity + 1):
            if weights[i - 1] <= w:
                dp[i][w] = max(values[i - 1] + dp[i - 1][w - weights[i - 1]],
                               dp[i - 1][w])
            else:
                dp[i][w] = dp[i - 1][w]
    return dp[n][capacity]
weights = [2, 3, 4, 5]
values = [3, 4, 5, 6]
capacity = 5
print("Weights :", weights)
print("Values  :", values)
print("Capacity:", capacity)
print("Maximum Profit:", knapsack(weights, values, capacity))
```

Sample Input:

Weights = [2, 3, 4, 5]

Values = [3, 4, 5, 6]

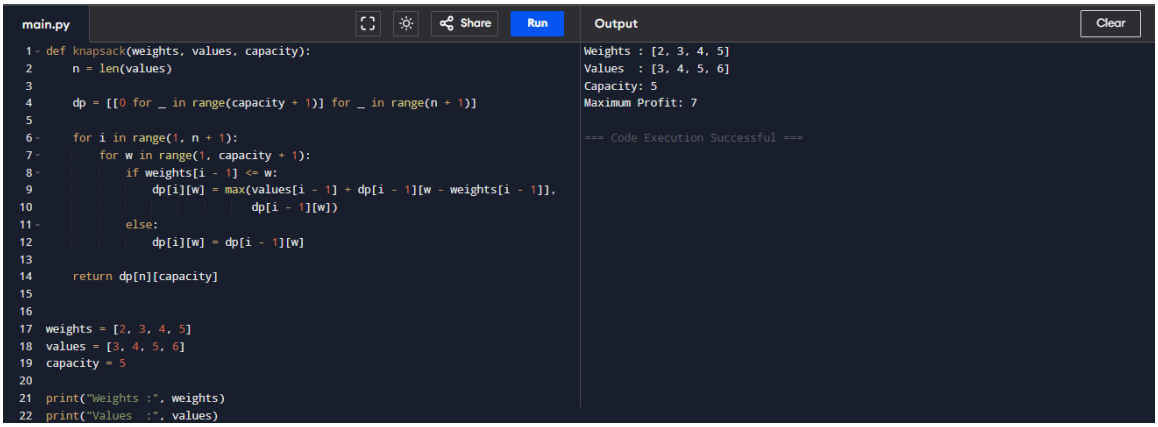
Capacity = 5

Sample Output:
Weights : [2, 3, 4, 5]

Values : [3, 4, 5, 6]

Capacity: 5

Maximum Profit: 7



Result:
The program was executed successfully and the expected output was obtained.

Experiment 12

Aim:
As specified in the uploaded experiment sheet.

Algorithm:
1. Read input.
2. Apply the required algorithm.
3. Display the result.

Pseudocode:
START
Read input
Process using algorithm
Print output
STOP

Python Program:

```
items = [
   (60, 10),
   (100, 20),
   (120, 30)
]

capacity = 50

items.sort(key=lambda x: x[0] / x[1], reverse=True)

profit = 0

for value, weight in items:
   if capacity >= weight:
      profit += value
      capacity -= weight
   else:
      profit += value * (capacity / weight)
      break

print("Items (Value, Weight):", items)
print("Maximum Profit:", profit)
```

Sample Input:
Items:

(60,10)

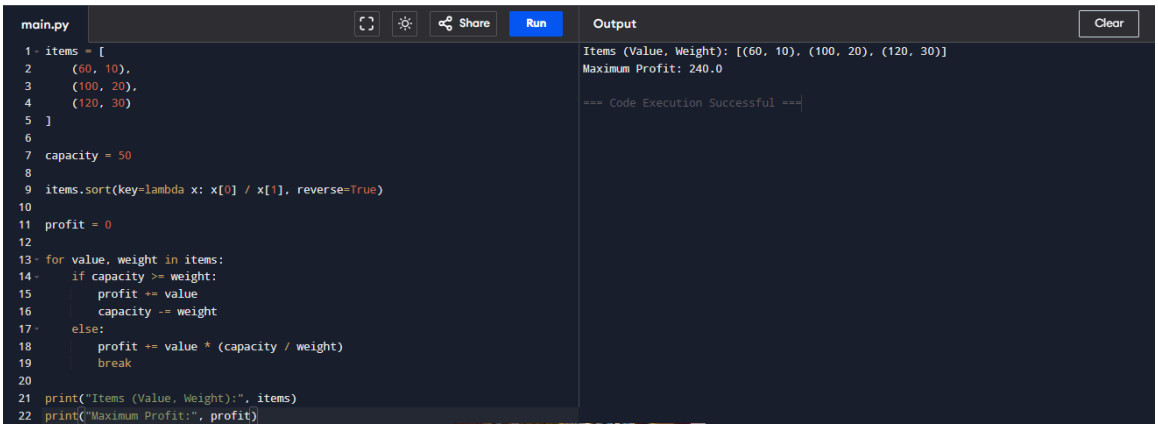
(100,20)

(120,30)

Capacity = 50

Sample Output:
Items (Value, Weight): [(60, 10), (100, 20), (120, 30)]

Maximum Profit: 240.0



Result:
The program was executed successfully and the expected output was obtained.

Experiment 13

Aim:
As specified in the uploaded experiment sheet.

Algorithm:
1. Read input.
2. Apply the required algorithm.
3. Display the result.

Pseudocode:
START
Read input
Process using algorithm
Print output
STOP

Python Program:

```
import heapq

text = "AAABBCCCCDD"

frequency = {}

for ch in text:

   frequency[ch] = frequency.get(ch, 0) + 1

heap = [[weight, [char, ""]] for char, weight in frequency.items()]

heapq.heapify(heap)

while len(heap) > 1:

   low = heapq.heappop(heap)

   high = heapq.heappop(heap)

   for pair in low[1:]:

      pair[1] = '0' + pair[1]

   for pair in high[1:]:

      pair[1] = '1' + pair[1]

   heapq.heappush(heap, [low[0] + high[0]] + low[1:] + high[1:])

codes = sorted(heapq.heappop(heap)[1:], key=lambda p: (len(p[1]), p))

print("Character\tCode")

for char, code in codes:

   print(char, "\t\t", code)
```

Sample Input:
Text = AAABBCCCCDD

Sample Output:
Character Code

C 0

A 10

B 110

D 111

main.py

```python
import heapq

text = "AAABBCCCCDD"

frequency = {}

for ch in text:
    frequency[ch] = frequency.get(ch, 0) + 1

heap = [[weight, [char, ""]] for char, weight in frequency.items()]
heapq.heapify(heap)

while len(heap) > 1:
    low = heapq.heappop(heap)
    high = heapq.heappop(heap)

    for pair in low[1:]:
        pair[1] = '0' + pair[1]

    for pair in high[1:]:
        pair[1] = '1' + pair[1]
```

Output

```
Character   Code
C           0
A           10
B           110
D           111

=== Code Execution Successful ===
```

Result:
The program was executed successfully and the expected output was obtained.

Experiment 14

Aim:
As specified in the uploaded experiment sheet.

Algorithm:
1. Read input.
2. Apply the required algorithm.
3. Display the result.

Pseudocode:
START
Read input
Process using algorithm
Print output
STOP

Python Program:

```
N = 4


board = [[0] * N for _ in range(N)]



def is_safe(row, col):
    for i in range(col):
        if board[row][i]:
            return False

    i, j = row, col
    while i >= 0 and j >= 0:
        if board[i][j]:
            return False
        i -= 1
        j -= 1

    i, j = row, col
    while i < N and j >= 0:
        if board[i][j]:
            return False
        i += 1
        j -= 1

    return True
```

```
def solve(col):
    if col >= N:
        return True

    for i in range(N):
        if is_safe(i, col):
            board[i][col] = 1

            if solve(col + 1):
                return True

            board[i][col] = 0

    return False



solve(0)

print("Solution for", N, "Queens:\n")

for row in board:
    print(row)
```

Sample Input:
Number of Queens = 4

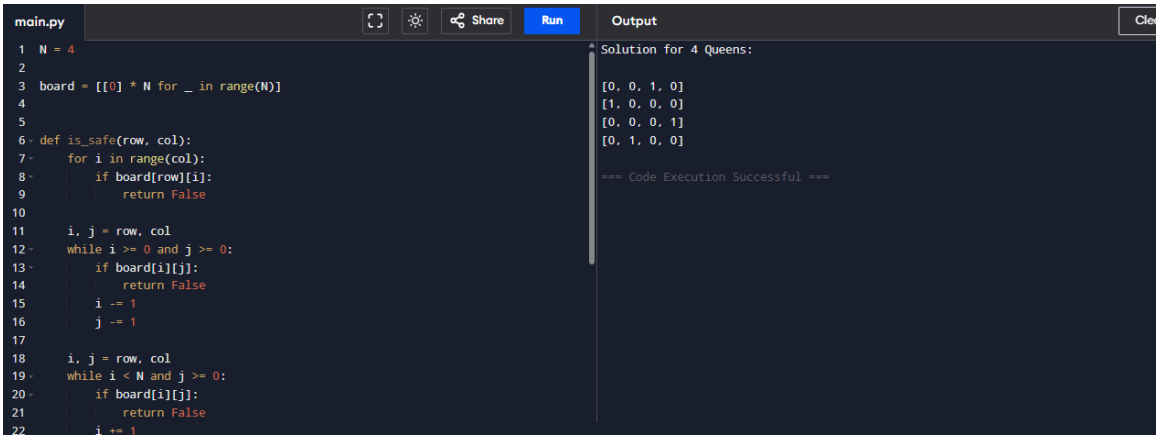
Sample Output:
Solution for 4 Queens:

[0, 0, 1, 0]

[1, 0, 0, 0]

[0, 0, 0, 1]

[0, 1, 0, 0]



Result:
The program was executed successfully and the expected output was obtained.